

UML - 02 - Modellazione della struttura

Francesco Betti Sorbelli

Università degli studi di Perugia. `francesco.bettisorbelli@unipg.it`



Oggetto

- Individui di un sistema

o:C

- Notazioni alternative:

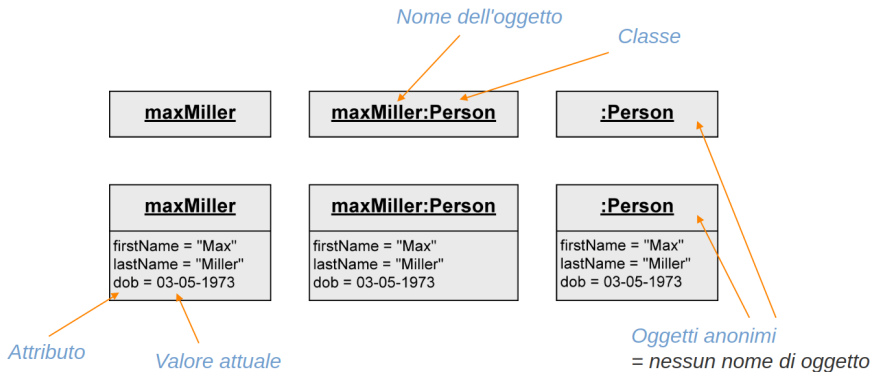
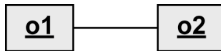
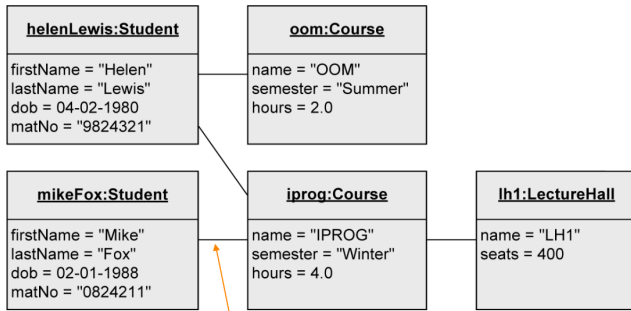


Diagramma degli oggetti

- Simbolo



- Oggetti di un sistema e loro relazioni (collegamenti)
- Istantanea di oggetti in un momento temporale specifico

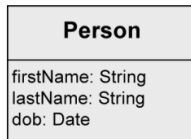


Collegamento

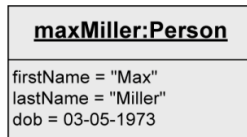
Da oggetto a classe

- Gli individui di un sistema hanno spesso caratteristiche e comportamenti identici.
- Una classe è un piano di costruzione per un insieme di oggetti simili di un sistema.
- Gli oggetti sono istanze di classi
- Attributi: caratteristiche strutturali di una classe
 - Valore diverso per ogni istanza (= oggetto)
- Operazioni: comportamento di una classe
 - Identico per tutti gli oggetti di una classe
 - Non rappresentati nel diagramma degli oggetti

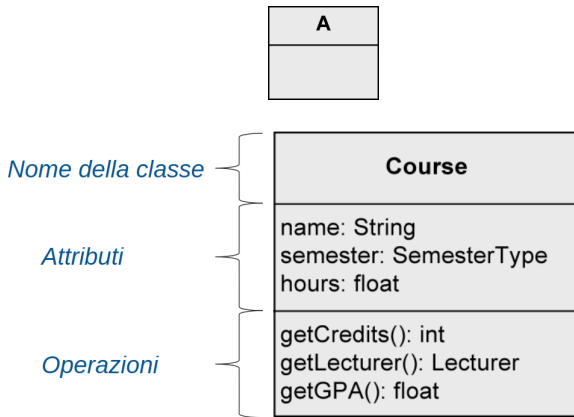
Classe



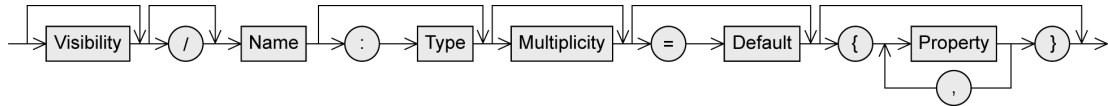
Oggetto di quella classe



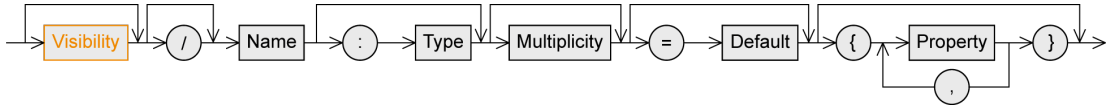
- Simbolo



Sintassi degli attributi



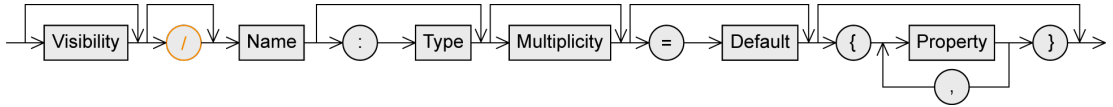
Sintassi degli attributi - Visibilità



Person
+ firstName: String + lastName: String - dob: Date # address: String[1..*] {unique, ordered} - ssNo: String {readOnly} - /age: int - password: String = "pw123" - <u>personsNumber: int</u>

- Chi può accedere all'attributo
 - + public: tutti
 - - private: solo l'oggetto stesso
 - # protected: classe stessa e sottoclassi
 - ~ package: classi che si trovano nello stesso package

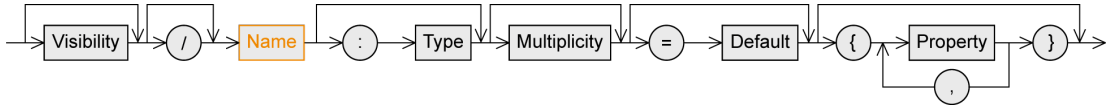
Sintassi dell'attributo - Attributo derivato



Person
firstName: String
lastName: String
dob: Date
address: String[1..*] {unique, ordered}
ssNo: String {readOnly}
age: int
password: String = "pw123"
<u>personsNumber: int</u>

- Il valore dell'attributo è derivato/dedotto da altri attributi (ridondante)
 - age: calcolata a partire dalla data di nascita (dob)

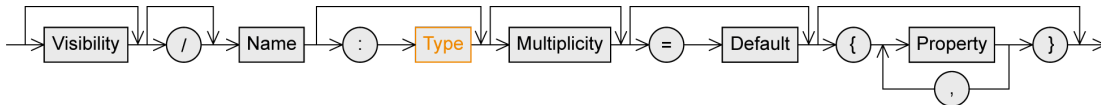
Sintassi dell'attributo - Nome



Person
<code>firstName: String</code> <code>lastName: String</code> <code>dob: Date</code> <code>address: String[1..*] {unique, ordered}</code> <code>ssNo: String {readOnly}</code> <code>/age: int</code> <code>password: String = "pw123"</code> <code><u>personsNumber: int</u></code>

- Nome dell'attributo

Sintassi dell'attributo - Tipo



Tipi:

- Classi definite dall'utente
- Tipo di dati
 - Tipo di dati primitivi
 - Predefiniti: Boolean, Integer, String
 - Definito dall'utente: "primitive"
 - Tipo di dati composito: "datatype"
 - Enumerazioni: "enumeration"

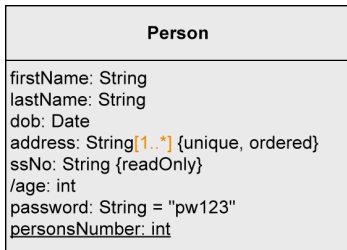
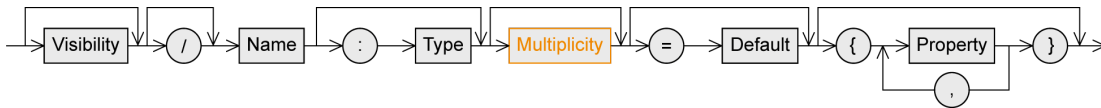
Person
firstName: String lastName: String dob: Date address: String[1..*] {unique, ordered} ssNo: String {readOnly} /age: int password: String = "pw123" <u>personsNumber: int</u>

«primitive» Float
round(): void

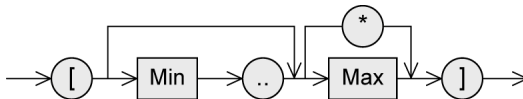
«datatype» Date
day month year

«enumeration» AcademicDegree
bachelor master phd

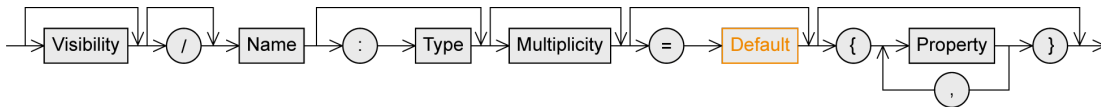
Sintassi degli attributi - Molteplicità



- Numero di valori che un attributo può contenere
- Valore predefinito: 1
- Notazione: $[min..max]$
 - nessun limite superiore: $[*]$ o $[0..*]$

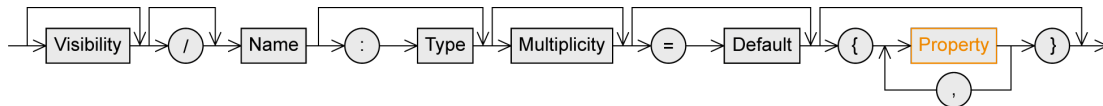


Sintassi dell'attributo - Valore predefinito



Person
firstName: String lastName: String dob: Date address: String[1..*] {unique, ordered} ssNo: String {readOnly} /age: int password: String = "pw123" <u>personsNumber: int</u>

- Valore predefinito
 - Utilizzato se il valore dell'attributo non è stato impostato esplicitamente dall'utente.



Person
firstName: String lastName: String dob: Date address: String[1..*] {unique, ordered} ssNo: String {readOnly} /age: int password: String = "pw123" <u>personsNumber: int</u>

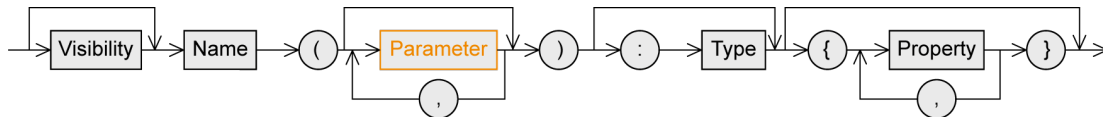
• Proprietà predefinite

- readOnly ... il valore non può essere modificato
- unique ... non sono consentiti duplicati
- non-unique ... sono consentiti i duplicati
- ordered ... ordine fisso dei valori
- unordered ... nessun ordine fisso dei valori

• Specifiche degli attributi

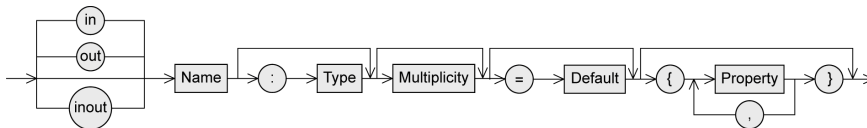
- Insiemi: unordered, unique
- Insiemi multipli: unordered, non-unique
- Insieme ordinato: ordered, unique
- Liste: ordered, non-unique

Sintassi dell'operazione - Parametri

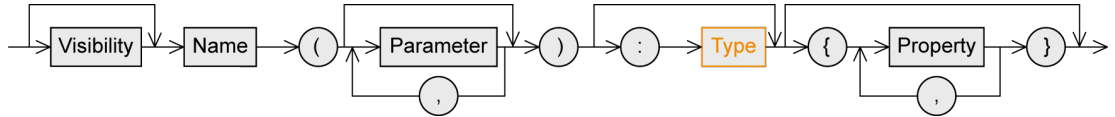


- Notazione simile a quella degli attributi
- Direzione del parametro
 - `in ...` parametro di ingresso
 - Quando si utilizza l'operazione, ci si aspetta un valore da questo parametro
 - `out ...` parametro di uscita
 - Dopo l'esecuzione dell'operazione, il parametro ha assunto un nuovo valore
 - `inout` : parametro combinato ingresso/uscita

Person
...
+ getName(out fn: String, out ln: String): void
+ updateLastName(newName: String): boolean
+ <u>getPersonsNumber(): int</u>



Sintassi dell'operazione - Tipo



Person
...
getName(out fn: String, out ln: String): void updateLastName(newName: String): boolean getPersonsNumber(): int

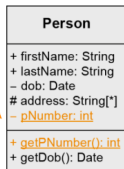
- Tipo del valore di ritorno

Variabili di classe e operazioni di classe

- Variabile di istanza (= attributo di istanza): attributi definiti a livello di istanza.
- Variabile di classe (= attributo di classe, attributo statico)
 - Definito una sola volta per classe, cioè condiviso da tutte le istanze della classe.
 - Ad esempio, contatori per il numero di istanze di una classe, costanti, ecc.
- Operazione di classe (= operazione statica)
 - Può essere utilizzato se non è stata creata un'istanza della classe corrispondente.
 - Ad esempio, costruttori, operazioni di conteggio, funzioni matematiche ($\sin(x)$), ecc.
- Notazione: sottolineatura del nome della variabile di classe / dell'operazione di classe

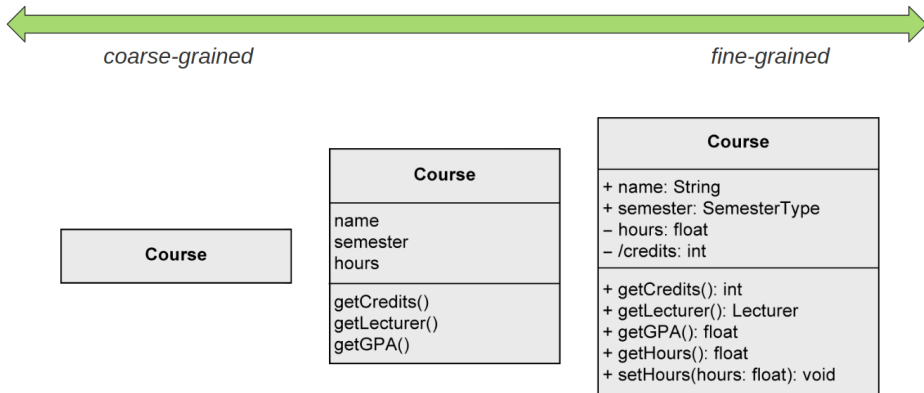
*Variabile
di classe*

*Operazioni
di classe*

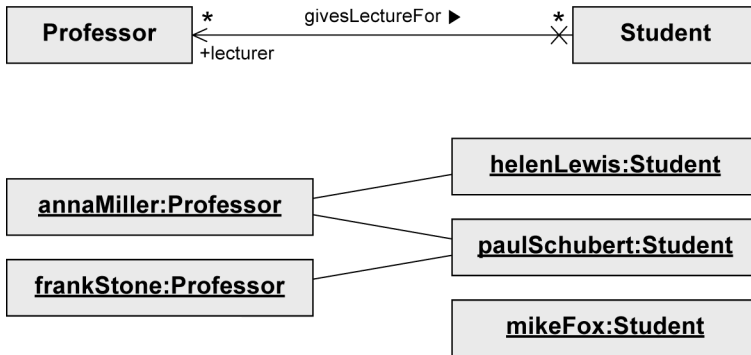


```
class Person {  
  
    public String firstName;  
    public String lastName;  
    private Date dob;  
    protected String[] address;  
    private static int pNumber;  
    public static int getPNumber() {...}  
    public Date getDob() {...}  
  
}
```

Specificazione delle classi: Diversi livelli di dettaglio

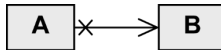


- Modelli di possibili relazioni tra istanze di classi

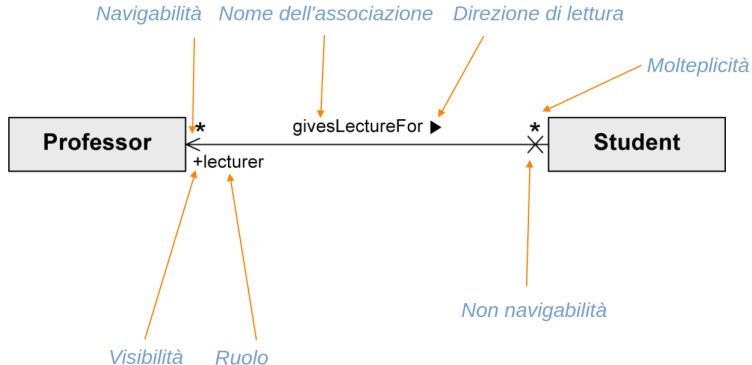


Associazione binaria

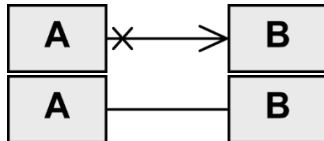
- Simbolo



- Collega tra loro le istanze di due classi

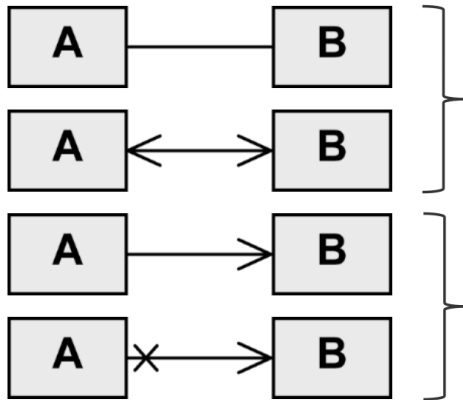


- Navigabilità: un oggetto conosce gli oggetti partner e può quindi accedere ai loro attributi e operazioni visibili.
 - Indicato dalla punta della freccia aperta
- Non navigabilità (esempio in alto)
 - Indicato da una croce
- Esempio
 - A può accedere agli attributi visibili e alle operazioni di B
 - B non può accedere a nessun attributo e operazioni di A
- Navigabilità non definita (esempio in basso)
 - Si presuppone la navigabilità bidirezionale

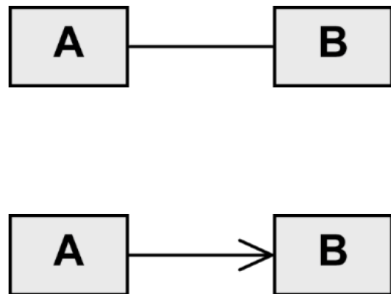


Navigabilità - Standard UML vs. Best Practice

UML standard

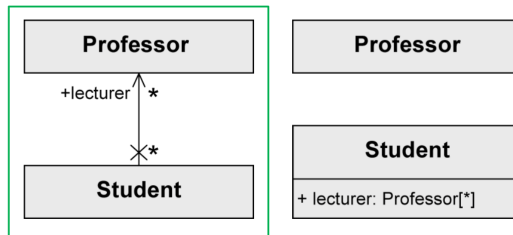


Best practice



Associazione binaria come attributo

Preferibile



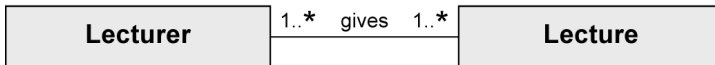
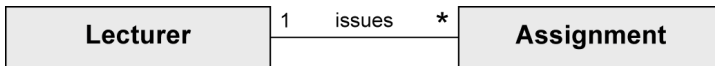
- Notazione simile a quella di Java:

```
class Professor {...}

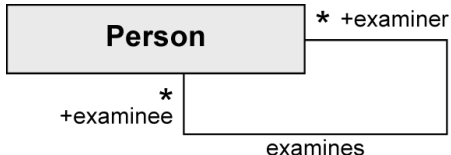
class Student{
    public Professor[] lecturer;
    ...
}
```

Associazione binaria - Molteplicità e ruolo

- Molteplicità: Numero di oggetti che possono essere associati esattamente a un oggetto del lato opposto.

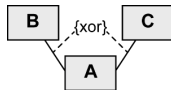


- Ruolo: descrive il modo in cui un oggetto è coinvolto in una relazione di associazione.



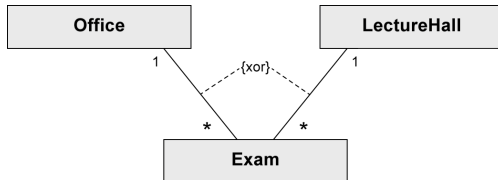
Associazione binaria - vincolo xor

- Simbolo

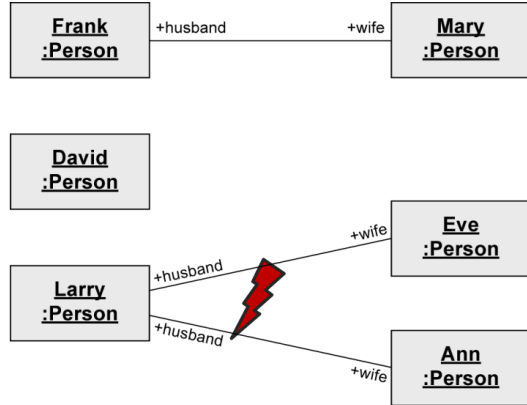
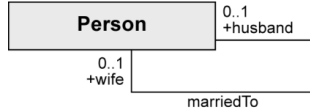


- Vincolo "exclusive or"

- Un oggetto di classe A deve essere associato a un oggetto di classe B o a un oggetto di classe C, ma non a entrambi.

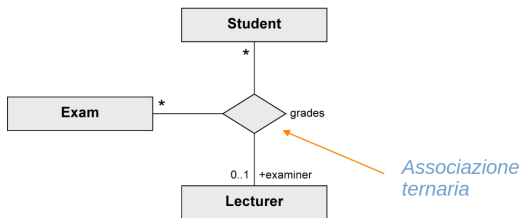


Associazione unaria - Esempio



Associazione n-aria (1/2)

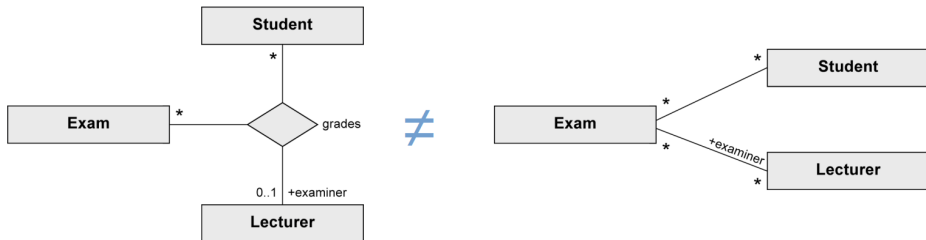
- Nella relazione sono coinvolti più di due oggetti partner.
- Si utilizza il simbolo del **rombo**.
- Nessuna indicazione di navigazione



Associazione n-aria (2/2)

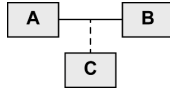
Esempio

- $(\text{Student}, \text{Exam}) \rightarrow (\text{Lecturer})$
 - Uno studente sostiene un esame con un solo o nessun docente (cioè non dà l'esame).
- $(\text{Exam}, \text{Lecturer}) \rightarrow (\text{Student})$
 - Un esame con un docente può essere sostenuto da un numero qualsiasi di studenti.
- $(\text{Student}, \text{Lecturer}) \rightarrow (\text{Exam})$
 - Uno studente può essere valutato da un `Lecturer` per un numero qualsiasi di esami.

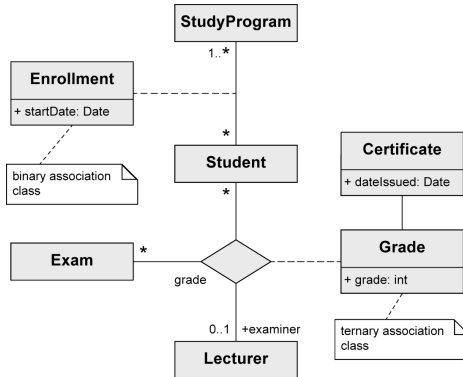


Classe di associazione

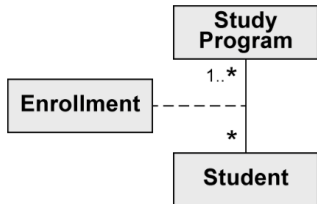
- Simbolo



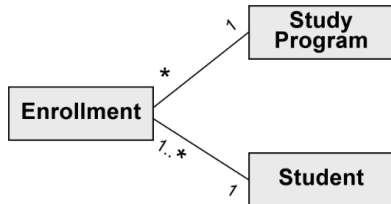
- Assegnare gli attributi alla **relazione** tra le classi piuttosto che alla **classe** stessa.



Classe di associazione vs. classe regolare



≠

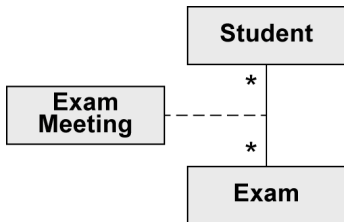


Uno studente può iscriversi a un particolare programma di studio solo **una volta**

Uno studente può avere **più** iscrizioni per uno e lo stesso programma di studio

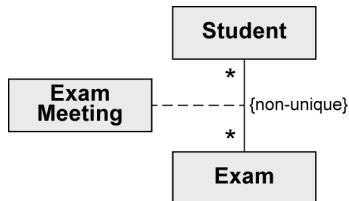
Classe di associazione - unique/non-unique (1/2)

- Predefinito: nessun duplicato (*unique*)



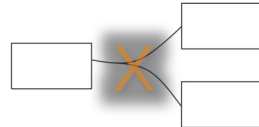
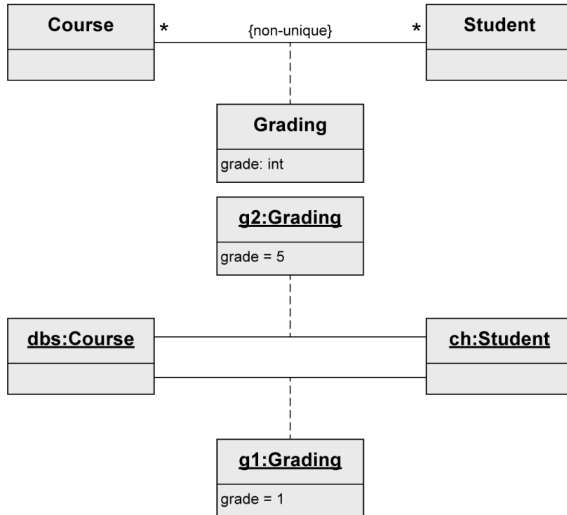
Uno studente può sostenere un **solo** orale di un'esame specifico.

- *non-unique*: sono consentiti i duplicati



Uno studente può sostenere **più** orali per un esame specifico.

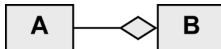
Classe di associazione - unique/non-unique (2/2)



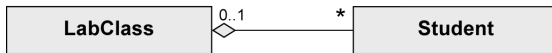
- Forma speciale di associazione
- Utilizzato per esprimere che una classe fa parte di un'altra classe.
- Proprietà dell'associazione di aggregazione:
 - Transitivo: se B fa parte di A e C fa parte di B, anche C fa parte di A.
 - Asimmetrico: non è possibile che A faccia parte di B e che B faccia parte di A contemporaneamente.
- Due tipi:
 - Aggregazione condivisa
 - Composizione

Aggregazione condivisa

- Simbolo



- Esprime una **appartenenza debole** delle parti a un insieme
 - = Le parti **esistono anche indipendentemente** dall'insieme
- La molteplicità all'estremità dell'aggregazione può essere > 1
 - = Un elemento può far parte simultaneamente di più elementi.
- Si estende su un grafo aciclico diretto
- Sintassi: Rombo vuoto all'estremità aggregante
- Esempio:
 - Student fa parte di LabClass
 - Course fa parte di StudyProgram



- Simbolo



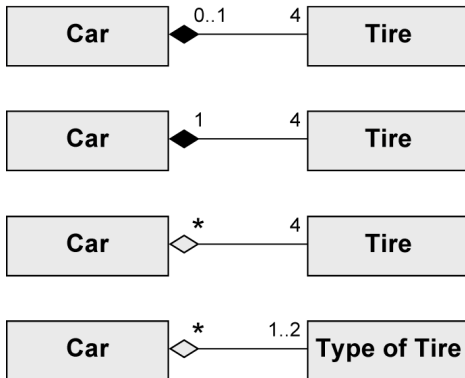
- Dipendenza dall'esistenza tra l'oggetto composito e le sue parti
- Una parte può essere contenuta al massimo in un oggetto composito in un determinato momento.
- Se l'oggetto composito viene eliminato, vengono eliminate anche le sue parti.
- Sintassi: Rombo pieno all'estremità dell'aggregazione
- Esempio: `Beamer` fa parte di `LectureHall` fa parte di `Building`



Se `Building` viene cancellato, anche `LectureHall` viene cancellata. Il `Beamer` può esistere senza la `LectureHall`, ma se è contenuto nella `LectureHall` mentre viene cancellato, il `Beamer` viene cancellato anche

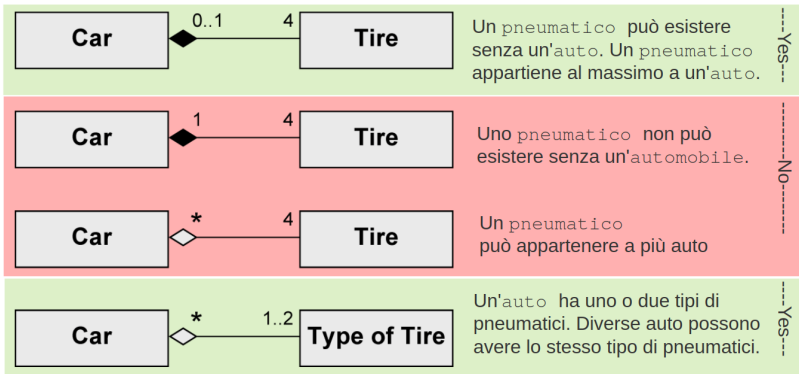
Aggregazione e composizione condivisa

- Quale modello si applica?

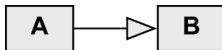


Aggregazione e composizione condivisa

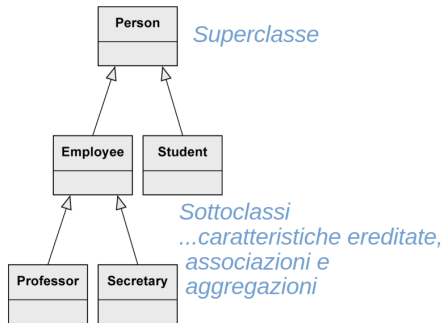
- Quale modello si applica?



- Simbolo



- Le caratteristiche (attributi e operazioni), le associazioni e le aggregazioni specificate per una classe generale (superclasse) vengono trasmesse alle sue sottoclassi.
- Ogni istanza di una sottoclasse è contemporaneamente un'istanza indiretta della superclasse.
- La sottoclasse eredita tutte le caratteristiche, le associazioni e le aggregazioni della superclasse, tranne quelle private.
- La sottoclasse può avere ulteriori caratteristiche, associazioni e aggregazioni.
- Le generalizzazioni sono transitive.



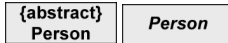
*Un Segretario è
un Dipendente e
a Persona*

Generalizzazione - Classe astratta

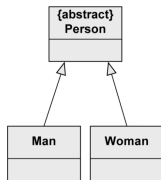
- Simbolo



- Utilizzati per evidenziare le caratteristiche comuni delle loro sottoclassi.
- Utilizzato per garantire che non vi siano istanze dirette della superclasse.



- Solo le sue sottoclassi non astratte possono essere istanziate.
- Utile nel contesto delle relazioni di generalizzazione.
- Notazione: parola chiave `abstract` o nome della classe in carattere corsivo.

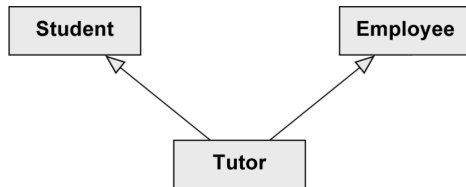


Nessun oggetto-persona possibile

Due tipi di persone: Uomo e Donna

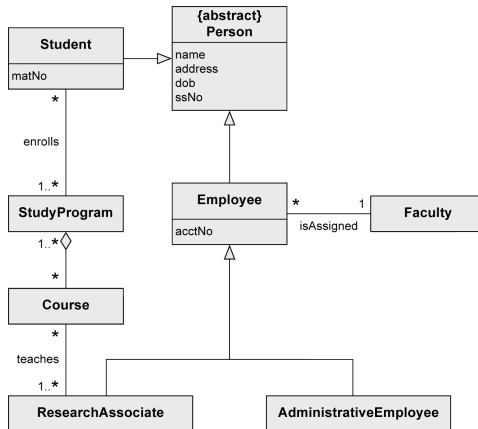
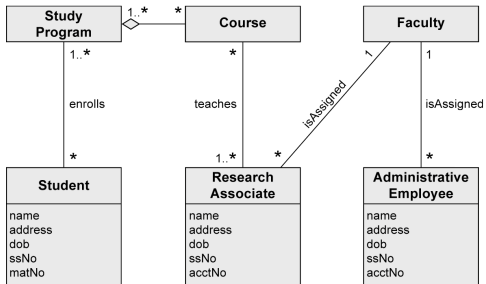
Generalizzazione - Ereditarietà multipla

- UML consente l'ereditarietà multipla.
- Una classe può avere più superclassi.
- Esempio:



Un tutor è sia un dipendente che uno studente

Con e senza generalizzazione



Creare un diagramma di classe

- Non è possibile estrarre completamente classi, attributi e associazioni da un testo in linguaggio naturale in modo automatico.
- Linee guida
 - I sostantivi spesso indicano classi
 - Gli aggettivi indicano i valori degli attributi
 - I verbi indicano operazioni
- Esempio: Il sistema di gestione della biblioteca memorizza gli utenti con il loro ID univoco, il nome e l'indirizzo, nonché i libri con il titolo, l'autore e il numero ISBN. Ann Foster vuole utilizzare la biblioteca.



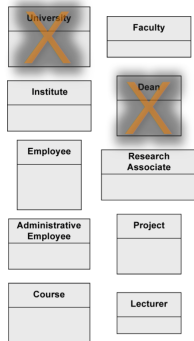
Esempio - Sistema informativo universitario

- Un'università è costituita da più facoltà che sono composte da vari istituti. Ogni facoltà e ogni istituto hanno un nome. Per ogni istituto è noto un indirizzo.
- Ogni facoltà è guidata da un preside, che è un dipendente dell'università.
- Il numero totale di dipendenti è noto. I dipendenti hanno un numero di previdenza sociale, un nome e un indirizzo e-mail. Esiste una distinzione tra personale di ricerca e personale amministrativo.
- I ricercatori sono assegnati ad almeno un istituto. Il campo di studio di ogni collaboratore di ricerca è noto. Inoltre, i ricercatori possono essere coinvolti in progetti per un certo numero di ore e sono noti il nome, la data di inizio e la data di fine dei progetti. Alcuni ricercatori tengono corsi. In questo caso sono chiamati docenti.
- I corsi hanno un numero unico (ID), un nome e una durata settimanale in ore.

Esempio - Fase 1: identificazione delle classi

- Un **università** è costituita da più **facoltà**, che sono composte da vari **istituti**. Ogni facoltà e ogni istituto hanno un nome. Per ogni istituto è noto un indirizzo.
- Ogni facoltà è guidata da un **preside**, che è un **dipendente** dell'università.
- Il numero totale di dipendenti è noto. I dipendenti hanno un numero di previdenza sociale, un nome e un indirizzo e-mail. Esiste una distinzione tra **personale di ricerca** e **personale amministrativo**.
- I ricercatori sono assegnati ad almeno un istituto. Il campo di studio di ogni collaboratore di ricerca è noto. Inoltre, i ricercatori possono essere coinvolti in **progetti** per un certo numero di ore e sono noti il nome, la data di inizio e la data di fine dei progetti. Alcuni ricercatori tengono **corsi**. In questo caso sono chiamati **docenti**.
- I corsi hanno un numero unico (ID), un nome e una durata settimanale in ore.

Modelliamo il sistema "Università"



Il preside non ha attributi ulteriori rispetto a qualsiasi altro dipendente

Esempio - Fase 2: identificazione degli attributi

- Un'università è costituita da più facoltà che sono composte da vari istituti. Ogni facoltà e ogni istituto hanno un **nome**. Per ogni istituto è noto un **indirizzo**.
- Ogni facoltà è guidata da un preside, che è un dipendente dell'università.
- Il **numero totale di dipendenti** è noto. I dipendenti hanno un **numero di previdenza sociale**, un **nome** e un **indirizzo e-mail**. Esiste una distinzione tra personale di ricerca e personale amministrativo.
- I ricercatori sono assegnati ad almeno un istituto. Il campo di studio di ogni collaboratore di ricerca è noto. Inoltre, I ricercatori possono essere coinvolti in progetti per un certo **numero di ore** e sono noti il **nome**, la **data di inizio** e la **data di fine** dei progetti. Alcuni ricercatori tengono corsi. In questo caso sono chiamati docenti.
- I corsi hanno un **numero unico (ID)**, un **nome** e una **durata settimanale** in ore.

Faculty
+ name: String

Institute
+ name: String
+ address: String

Employee
+ ssNo: int
+ name: String
+ email: String
+ counter: int

Research Associate
+ fieldOfStudy: String

Administrative Employee

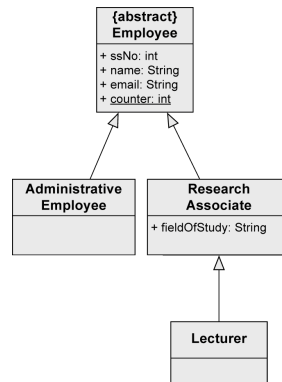
Project
+ name: String
+ start: Date
+ end: Date

Course
+ name: String
+ id: int
+ hours: float

Lecturer

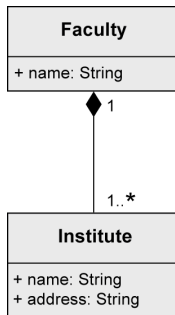
Esempio - Fase 2: Identificare le relazioni (1/6)

- Tre tipi di relazioni:
 - Associazione
 - Generalizzazione
 - Aggregazione
- `abstract`, cioè nessun altro tipo di dipendenti
- **Generalizzazioni:**
- "C'è una distinzione tra personale di ricerca e personale amministrativo".
- "Alcuni ricercatori tengono dei corsi. Poi vengono chiamati docenti".



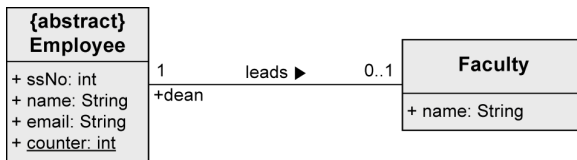
Esempio - Fase 2: Identificazione delle relazioni (2/6)

- "Un'università è costituita da più facoltà che sono composte da vari istituti".
- **Composizione** per dimostrare l'esistenza della dipendenza



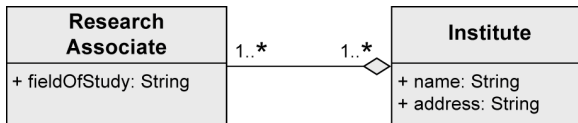
Esempio - Fase 2: Identificazione delle relazioni (3/6)

- "Ogni facoltà è guidata da un preside, che è un dipendente dell'università".
- Nella **relazione** di leadership, il dipendente assume il ruolo di preside (dean).



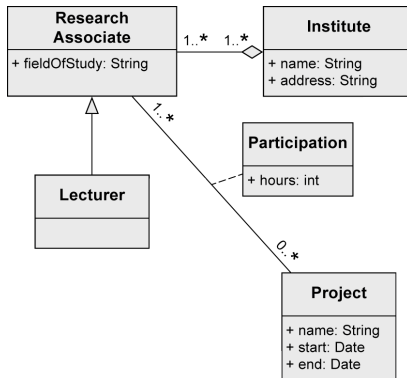
Esempio - Fase 2: Identificazione delle relazioni (4/6)

- "I ricercatori sono assegnati ad almeno un istituto".
- **Aggregazione condivisa** per mostrare che i `ResearchAssociate` fanno parte di un `Institute`, ma non c'è dipendenza dall'esistenza



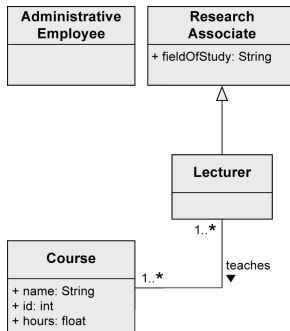
Esempio - Fase 2: Identificazione delle relazioni (5/6)

- "Inoltre, i ricercatori possono essere coinvolti in progetti per un certo numero di ore".
- **L'associazione di classe** *Participation* permette di memorizzare il numero di ore per ogni singolo *Project* di ogni singolo *ResearchAssociate*

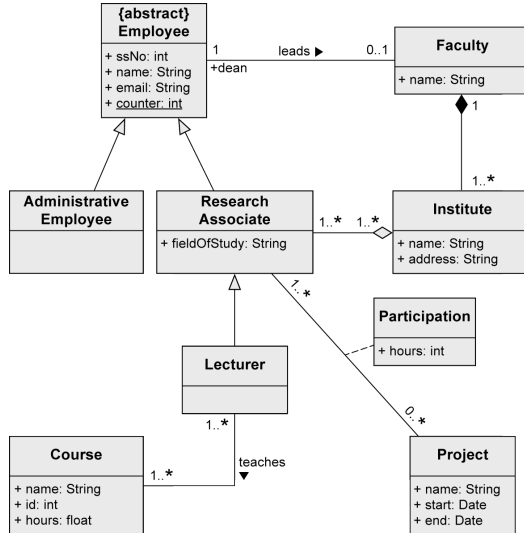


Esempio - Fase 2: Identificare le relazioni (6/6)

- "Alcuni ricercatori tengono dei corsi. Poi vengono chiamati docenti".
- `Lecturer` eredita tutte le caratteristiche, associazioni e aggregazioni da `ResearchAssociate`
- Inoltre, un `Lecturer` ha un'**associazione** `teaches` al `Course`.

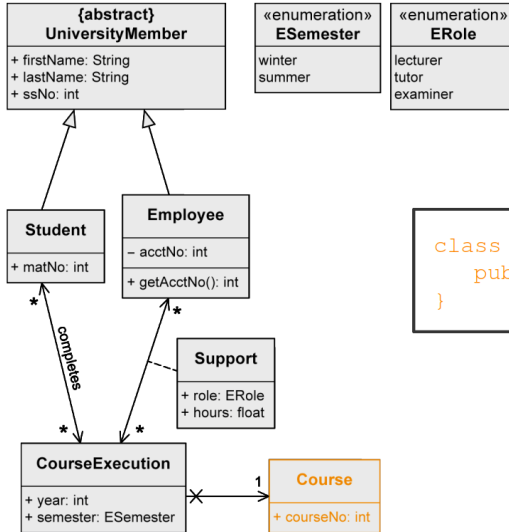


Esempio - Diagramma di classe completo



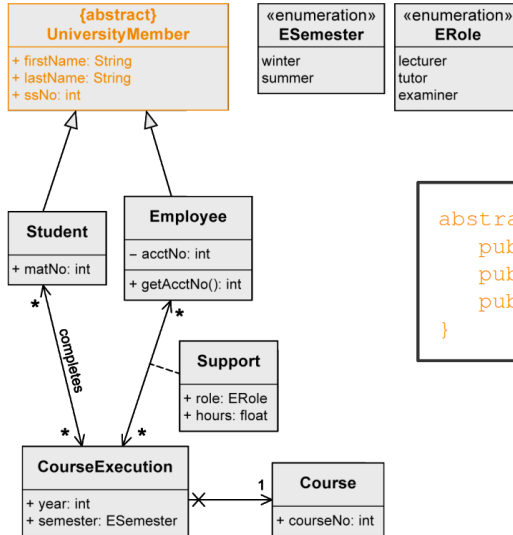
- I diagrammi delle classi sono spesso creati con l'intenzione di implementare gli elementi modellati in un linguaggio di programmazione orientato agli oggetti.
- Spesso la traduzione è semiautomatica e richiede solo un minimo intervento manuale.

Generazione del codice - Esempio (1/6)



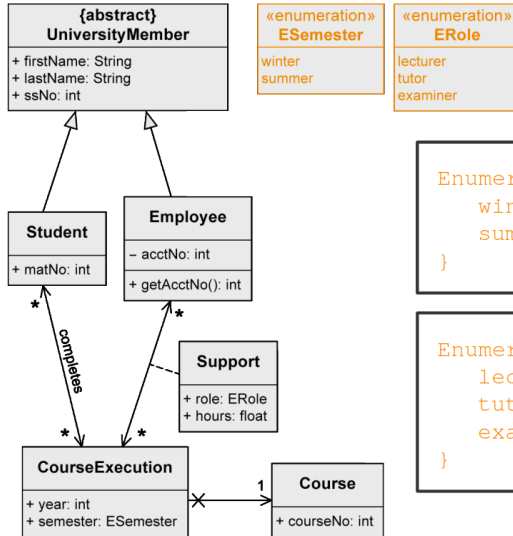
```
class Course {  
    public int courseNo;  
}
```

Generazione del codice - Esempio (2/6)



```
abstract class UniversityMember {
    public String firstName;
    public String lastName;
    public int ssNo;
}
```

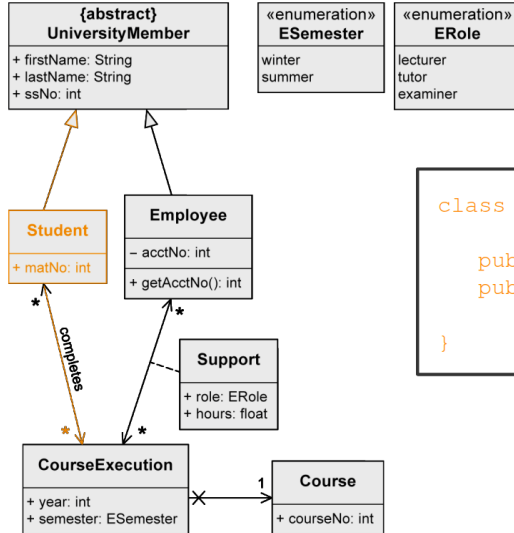
Generazione del codice - Esempio (3/6)



```
Enumeration ESemester {
    winter,
    summer
}
```

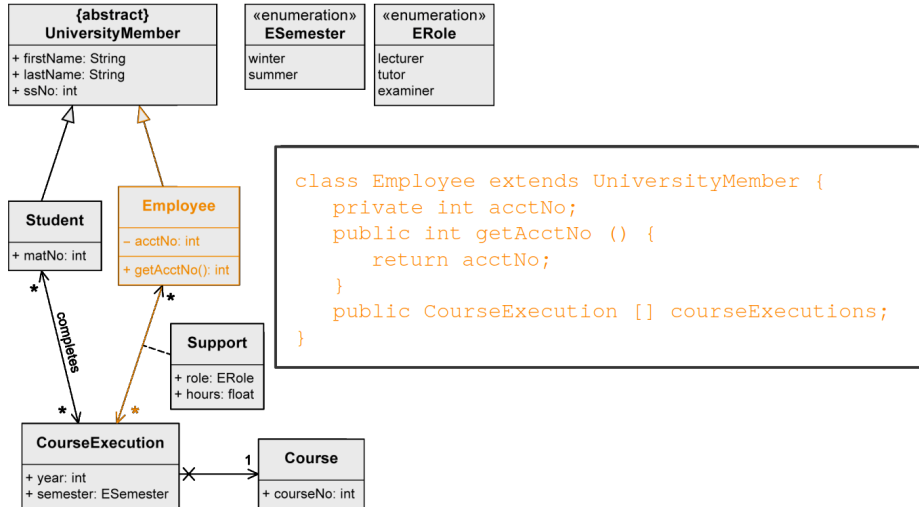
```
Enumeration ERole {
    lecturer,
    tutor,
    examiner
}
```


Generazione del codice - Esempio (4/6)

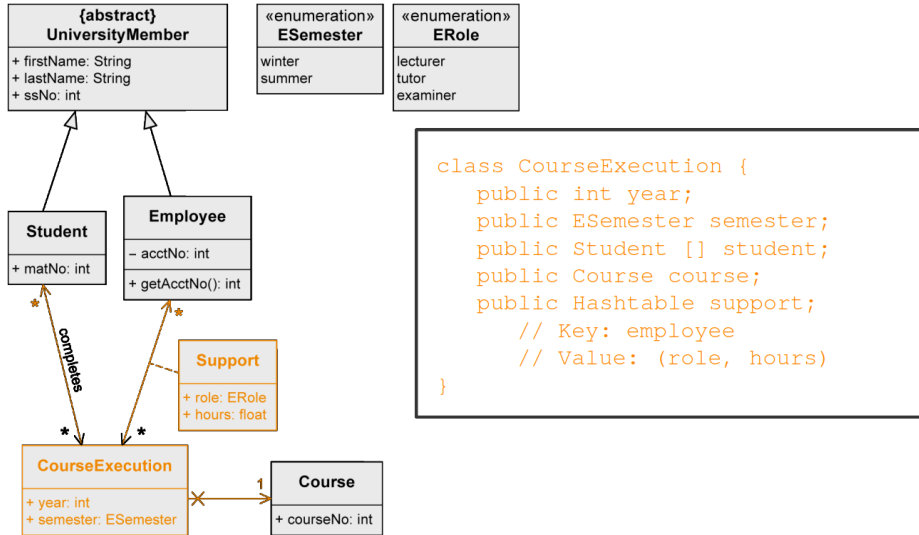


```
class Student extends
    UniversityMember {
    public int matNo;
    public CourseExecution []
        completedCourses;
}
```

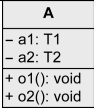
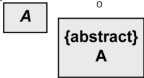
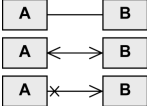
Generazione del codice - Esempio (5/6)



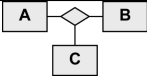
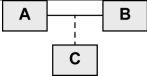
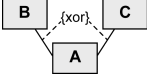
Generazione del codice - Esempio (6/6)



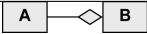
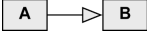
Elementi di notazione (1/3)

Nome	Notazione	Descrizione
Classe		Descrizione della struttura e del comportamento di un insieme di oggetti
Classe astratta		Classe che non può essere istanziata
Associazione		Relazione tra le classi: navigabilità non specificata, navigabile in entrambe le direzioni, non navigabile in una sola direzione

Elementi di notazione (2/3)

Nome	Notazione	Descrizione
Associazione n-aria		Relazione tra n (qui 3) classi
Classe di associazione		Descrizione più dettagliata di un'associazione
Relazione xor		Un oggetto di C è in relazione con un oggetto di A o con un oggetto di B, ma non con entrambi.

Elementi di notazione (3/3)

Nome	Notazione	Descrizione
Aggregazione condivisa		Relazione parte-intero (A è parte di B)
Aggregazione forte = Composizione		Relazione esistenza-dipendente parti-intero (A è parte di B)
Generalizzazione		Relazione di eredità (A eredita da B)
Oggetto		Istanza di una classe
Collegamento		Relazione tra gli oggetti